

POWERED SHOPPING - CURRENT PROJECT REPORT

Generated on: 2026-04-20 09:22 IST

Project root: /Users/ayazmallick/Desktop/Projects/Powered Shopping

1) EXECUTIVE SUMMARY

Powered Shopping is a full-stack voice-enabled shopping assistant with a React/Vite frontend and an Express backend. The frontend supports Firebase authentication, voice command capture, typed command fallback, and a multi-view shopping dashboard. The backend provides product browsing, recommendations, cart management, AI assistant tool-calling, and audio transcription endpoints.

The codebase is functional and has clear fallbacks for external dependencies:

- Product API falls back to local mock catalog data.
- Cart storage falls back to in-memory storage when Firebase is unavailable.
- Voice command parsing falls back to a deterministic intent parser when AI assistant mode is unavailable.
- Browser speech recognition has a fallback recording/transcription flow.

2) PROJECT SNAPSHOT

- Type: Monorepo-style full-stack JavaScript application
- Frontend: React 18 + Vite 5 + Firebase SDK
- Backend: Node.js + Express 4
- Auth: Firebase Auth (email/password + Google sign-in)
- AI: OpenAI Chat Completions tool-calling + Whisper transcription API
- Source files analyzed (excluding node_modules/dist): 39
- Approximate source LOC analyzed: 7,414

Top-level directories:

- client/
- server/
- node_modules/ (root)

3) FRONTEND ARCHITECTURE (CLIENT/)

3.1 APP BOOTSTRAPPING

Entry point: client/src/main.jsx

- Initializes analytics via 'initAnalytics()'.
- Mounts 'AppRoot'.
- Uses 'useFirebaseAuth' hook to gate app access:
- Loading state: shows session preparation card.
- Unauthenticated: renders 'AuthScreen'.
- Authenticated: renders 'HomePage'.

3.2 CORE UI ORCHESTRATION

Primary container: client/src/pages/HomePage.jsx

Key responsibilities:

- Controls view routing through local state ('overview', 'shop', 'cart', 'checkout', 'assistant', 'settings', 'faq').
- Maintains product list, recommendation list, cart summary, history, voice state, and user settings.
- Persists user preferences via localStorage:
 - theme
 - assistant settings
 - last used category
- Integrates both AI assistant mode and deterministic intent mode.

3.3 VOICE AND COMMAND HANDLING

- 'useSpeechRecognition':
 - Uses Web Speech API with secure-context checks.
 - Handles continuous mode and retry for transient network errors.
 - Provides descriptive error messages for no-speech, not-allowed, audio-capture, network, etc.
- 'VoiceControl' component:
 - Supports live microphone commands.
 - Supports manual text commands.
 - Supports fallback audio recording via 'MediaRecorder' and backend transcription.
- 'useSpeechSynthesis':
 - Provides spoken responses for assistant outputs.

3.4 DATA ACCESS LAYER

File: client/src/services/api.js

Frontend consumes backend routes under '/api':

- product listing/filtering/categories/recommendations
- cart CRUD/checkout
- AI chat
- audio transcription

3.5 AI + INTENT FALLBACK ON CLIENT

- Primary mode ('settings.aiAssistantMode=true'):
 - Sends message + limited history to '/api/ai/chat'.
 - Applies returned UI patch objects ('products', 'recommendations', 'cart', 'orderResult', 'activeView', 'lastCategory').
- Fallback mode:
 - Uses 'detectIntent()' from 'intentParser.js'.
 - Handles intents: SEARCH, TRENDING, SIMILAR, ADD, REMOVE, CHECKOUT, HELP, UNKNOWN.

4) BACKEND ARCHITECTURE (SERVER/)

4.1 SERVER BOOTSTRAP

Entry point: server/index.js

- Loads '.env' values using custom loader ('loadLocalEnv').
- Configures 'cors' and JSON body parsing (12 MB limit).
- Health endpoint: 'GET /api/health'.
- Mounts route groups:
- '/api/products'
- '/api/cart'
- '/api/ai'
- Global error middleware returns standardized 500 response.

4.2 PRODUCT SERVICE

File: server/services/productService.js

- Primary catalog source: 'https://fakestoreapi.com/products'.
- Timeout: 3 seconds via AbortSignal.
- Fallback: local JSON ('server/data/mockProducts.json').
- Normalizes products:
- Keeps USD price as 'price'.
- Computes INR approximation as 'priceInr' (USD * 83).
- Normalizes rating object.
- Uses in-memory cache with 5-minute TTL.

4.3 RECOMMENDATION SERVICE

File: server/services/recommendationService.js

- Trending: ranked by 'rating.rate * rating.count'.
- Similar: same category, excludes current item when provided, sorted by rating.

4.4 CART PERSISTENCE AND STORAGE STRATEGY

File: server/services/cartStore.js

- Supports Firestore-backed cart storage when Firebase Admin is configured.
- Falls back to in-memory 'Map' when Firebase config/package is unavailable.
- Cart keying strategy supports:
- 'x-cart-id' header
- query/body 'cartId'
- 'DEFAULT_CART_ID'
- fallback 'default'
- Operations:
- get items
- add item (increment quantity when existing)
- remove by product ID
- remove by title query
- clear cart

4.5 AI ASSISTANT SERVICE

File: server/services/aiAssistantService.js

- OpenAI endpoints used:
- Chat: '/v1/chat/completions'
- Transcription: '/v1/audio/transcriptions'
- Default models:
- chat: 'gpt-4o-mini'
- transcription: 'whisper-1'
- Implements tool-calling loop ('TOOL_LOOP_LIMIT=6') with server-side executable tools:
- 'search_products'
- 'get_recommendations'
- 'add_to_cart'
- 'remove_from_cart'
- 'get_cart'
- 'checkout'
- Returns both assistant 'reply' and UI payload ('ui') for direct frontend state updates.

4.6 ROUTES AND CONTROLLERS

Routes (from server/routes):

- Products
- 'GET /api/products'
- 'GET /api/products/categories'
- 'GET /api/products/recommendations'
- Cart
- 'GET /api/cart'
- 'POST /api/cart/add'
- 'POST /api/cart/remove'
- 'POST /api/cart/checkout'
- AI
- 'POST /api/ai/chat'
- 'POST /api/ai/transcribe'

5) RUNTIME FLOW SUMMARY

5.1 VOICE/COMMAND FLOW

1. User speaks command or types text in assistant panel.
2. Client processes command through AI mode first (if enabled).
3. Backend AI tool-calls execute business operations (search/cart/recommend/checkout).
4. Backend returns natural-language reply + structured UI deltas.
5. Client updates product/cart/recommendation/view state and logs interaction.
6. If AI mode fails/unavailable, deterministic intent parser executes local fallback logic.

5.2 CATALOG RESILIENCE FLOW

1. Backend tries FakeStore API.
2. On timeout/error, backend loads local mock products.
3. Frontend additionally has local demo product fallback if API requests fail.

5.3 CART PERSISTENCE FLOW

1. Backend attempts Firestore initialization from env config.
2. If unavailable, warnings logged and in-memory cart mode activated.
3. Client behavior remains consistent regardless of storage backend.

6) CONFIGURATION AND ENVIRONMENT

6.1 BACKEND ('SERVER/.ENV')

Expected keys:

- 'PORT'
- 'DEFAULT_CART_ID'
- 'FIREBASE_PROJECT_ID'
- 'FIREBASE_CLIENT_EMAIL'
- 'FIREBASE_PRIVATE_KEY'
- 'OPENAI_API_KEY'
- 'OPENAI_MODEL'
- 'OPENAI_TRANSCRIBE_MODEL'

6.2 FRONTEND ('CLIENT/.ENV')

Expected keys:

- 'VITE_FIREBASE_API_KEY'
- 'VITE_FIREBASE_AUTH_DOMAIN'
- 'VITE_FIREBASE_PROJECT_ID'
- 'VITE_FIREBASE_STORAGE_BUCKET'
- 'VITE_FIREBASE_MESSAGING_SENDER_ID'
- 'VITE_FIREBASE_APP_ID'
- 'VITE_FIREBASE_MEASUREMENT_ID'

7) BUILD AND RUN

Root scripts ('package.json'):

- 'npm run dev' => starts server + client
- 'npm run dev:server'
- 'npm run dev:client'
- 'npm run build' => client build
- 'npm run start' => server start

Backend scripts ('server/package.json'):

- 'npm run dev' => nodemon on port 5001
- 'npm run start' => node on port 5001

8) CURRENT RISKS AND GAPS

1. Secret exposure risk in env examples:

- 'server/.env.example' and 'client/.env.example' currently contain what look like populated API/Firebase values, not placeholders.
- Recommendation: replace with placeholders and rotate any exposed credentials.

2. Port documentation mismatch:

- README states server at 'http://localhost:5000'.
- Current server scripts/defaults run on port '5001'.
- Recommendation: align README with code.

3. No automated test suite observed:

- No 'test' scripts in root/server/client package manifests.
- Recommendation: add baseline unit/integration coverage for intent parser, product filtering, cart operations, and AI route contract checks.

4. OpenAI API compatibility risk:

- AI service currently calls '/v1/chat/completions' and hardcodes 'gpt-4o-mini' default.
- Recommendation: keep model default configurable and verify endpoint/model compatibility during upgrades.

9) SUGGESTED NEXT ACTIONS

- Security hardening:
 - sanitize env example files
 - rotate leaked credentials
- Reliability:
 - add automated tests for key business flows
 - add structured request/error logging
- Product quality:
 - synchronize README with runtime behavior (ports and setup)
 - add API contract documentation examples for each endpoint

End of report.